

SQL Complete Practice Guide

Data Analyst Roadmap: Days 16-30 (SQL Beginner → Advanced)

Day	Topics	Questions
Day 16	Install MySQL, SELECT, DISTINCT	20
Day 17	WHERE, AND, OR, NOT, LIKE	25
Day 18	ORDER BY, LIMIT, BETWEEN, IN	20
Day 19	COUNT, SUM, AVG, MIN, MAX	20
Day 20	GROUP BY, HAVING	20
Day 21	INNER JOIN	20
Day 22	LEFT, RIGHT, FULL JOIN	20
Day 23	SELF JOIN, CROSS JOIN	15
Day 24	Subqueries	20
Day 25	CASE WHEN	15
Day 26	CTE (Common Table Expressions)	15
Day 27	ROW_NUMBER, RANK, DENSE_RANK	20
Day 28	LAG, LEAD	15
Day 29	Mixed Revision (All Topics)	50
Day 30	SQL Mini Project (Supermarket Sales)	32
TOTAL		327+ Questions

How to Use This Guide: Practice one day at a time. Solve without looking at solutions. Run every query yourself in MySQL Workbench, debug errors, and understand why they happened.

Recommended Time: 30-40 minutes daily for each day's questions. Days 29-30 are heavier (revision + project) and may need 60-90 minutes.

DAY 16: Install MySQL, SELECT, DISTINCT (20 Questions)

Q1: Install MySQL Community Server and MySQL Workbench, then connect to your local instance.

Q2: Create a database named 'company' using CREATE DATABASE.

Q3: Create an Employee table with columns EmpID, Name, Department, City, Salary.

Q4: Insert 10 sample rows into the Employee table using INSERT INTO.

Q5: Write a query to select all columns and all rows from Employee using SELECT *.

Q6: Select only the Name and Salary columns from Employee.

Q7: Select all unique Department values from Employee using SELECT DISTINCT.

Q8: Select all unique combinations of Department and City together.

Q9: Select all unique City values where employees are based.

Q10: Count how many distinct departments exist using COUNT(DISTINCT Department).

Q11: Select Name and Department, aliasing them as EmployeeName and Team using AS.

Q12: Use DESCRIBE Employee to view the table's column structure.

Q13: Select all columns from Employee but show only the first 5 rows using LIMIT.

Q14: Create a Department table with DeptID and DeptName columns.

Q15: Select DISTINCT Salary values from Employee to check for repeated pay values.

Q16: Add a calculated column to your SELECT: Salary * 12 AS AnnualSalary.

Q17: Add a new column Age to the Employee table using ALTER TABLE.

Q18: Update every row to set a default Age value using UPDATE ... SET.

Q19: Select DISTINCT City values and manually note how many employees are in each.

Q20: Write one query combining SELECT and DISTINCT to list every unique Department, in the order MySQL returns them.

DAY 17: WHERE, AND, OR, NOT, LIKE (25 Questions)

Q1: Select all customers who live in 'Mumbai' using WHERE.

Q2: Select all customers whose Age is greater than 30.

Q3: Select all customers who are NOT from 'Delhi' using NOT.

Q4: Select customers from 'Mumbai' AND with Age greater than 25.

Q5: Select customers from 'Mumbai' OR 'Pune'.

Q6: Select customers whose Name starts with 'A' using LIKE 'A%'.

Q7: Select customers whose Name ends with 'a' using LIKE '%a'.

Q8: Select customers whose Name contains 'an' anywhere using LIKE '%an%'.

Q9: Select customers whose Email ends with '@gmail.com'.

Q10: Select customers with Age between 20 and 40 using AND (without BETWEEN).

Q11: Select customers who are NOT from 'India'.

Q12: Select customers from 'Mumbai' AND NOT older than 50.

Q13: Select customers whose City is 'Mumbai', 'Pune', or 'Delhi' using multiple OR conditions.

Q14: Select customers whose Name is exactly 5 characters long using LIKE '_____ '.

Q15: Select customers where Age IS NULL.

Q16: Select customers where Age IS NOT NULL.

Q17: Select customers from 'Mumbai' with Age > 25, OR from 'Pune' with Age > 30.

Q18: Select customers whose second letter of the name is 'a' using LIKE '_a%'.

Q19: Select customers who do NOT live in 'Mumbai' or 'Pune'.

Q20: Select customers where Country is 'India' AND City is NOT 'Delhi'.

Q21: Select customers whose Email does not contain 'yahoo'.

Q22: Select customers where Age is greater than 18, less than 60, AND City is 'Mumbai'.

Q23: Select customers whose name starts with 'S' and ends with 'h'.

Q24: Select all customers except those whose Age is exactly 25.

Q25: Combine WHERE, AND, OR, NOT, and LIKE in one query: customers from 'Mumbai' or 'Pune', aged above 21, whose name does not start with 'Z'.

DAY 18: ORDER BY, LIMIT, BETWEEN, IN (20 Questions)

- Q1:** Select all products ordered by Price in ascending order.
- Q2:** Select all products ordered by Price in descending order.
- Q3:** Select the top 10 most expensive products using ORDER BY and LIMIT.
- Q4:** Select the 5 cheapest products.
- Q5:** Select products ordered by Category, then by Price within each category.
- Q6:** Select products with Price between 500 and 2000 using BETWEEN.
- Q7:** Select products whose Category is in ('Electronics', 'Furniture', 'Clothing') using IN.
- Q8:** Select products that are NOT IN the 'Clothing' category.
- Q9:** Select products with Stock between 10 and 50.
- Q10:** Select the 3 products with the lowest stock.
- Q11:** Select products ordered alphabetically by ProductName.
- Q12:** Select 10 products while skipping the first 5, using LIMIT with OFFSET.
- Q13:** Select products priced between 1000 and 5000, in Category IN ('Electronics', 'Mobiles').
- Q14:** Select products with Price NOT BETWEEN 100 and 500.
- Q15:** Select the 2nd highest-priced product using ORDER BY with LIMIT and OFFSET.
- Q16:** Select all products ordered by Stock descending, then Price ascending as a tiebreaker.
- Q17:** Select products whose ProductID is IN a specific list of 5 IDs you choose.
- Q18:** Select the 10 most recently added products using ORDER BY DateAdded DESC LIMIT 10.
- Q19:** Select the top 10 products from Category IN ('Electronics', 'Mobiles') ordered by Price DESC - today's main task.
- Q20:** Select the bottom 5 products by Stock for a restocking report.

DAY 19: COUNT, SUM, AVG, MIN, MAX (20 Questions)

- Q1:** Count the total number of rows in the Sales table using COUNT(*).
- Q2:** Count how many distinct products have been sold using COUNT(DISTINCT ProductName).
- Q3:** Find the total revenue using SUM(Amount).
- Q4:** Find the average sale amount using AVG(Amount).
- Q5:** Find the minimum sale amount using MIN(Amount).
- Q6:** Find the maximum sale amount using MAX(Amount).
- Q7:** Count how many sales happened in the 'Electronics' category.
- Q8:** Find the total quantity sold across all sales using SUM(Quantity).
- Q9:** Find the average quantity sold per sale.
- Q10:** Find the single highest sale amount and identify which product it belongs to.
- Q11:** Count how many distinct customers have placed at least one order.
- Q12:** Calculate total revenue for one specific category using WHERE and SUM together.
- Q13:** Find the minimum and maximum product price in a single query.
- Q14:** Count how many products have a Price greater than 1000.
- Q15:** Calculate the average salary of employees using AVG(Salary) on the Employee table.
- Q16:** Find the total salary expense for the company using SUM(Salary).
- Q17:** Count how many employees earn more than 50000 using COUNT with WHERE.
- Q18:** Find the number of orders placed after a specific date.
- Q19:** Calculate total revenue and average revenue per sale together in one summary query.
- Q20:** Build a one-row sales summary: total sales, total revenue, average sale, min sale, max sale - today's main task.

DAY 20: GROUP BY, HAVING (20 Questions)

Q1: Count the number of employees in each department using GROUP BY.

Q2: Find total salary paid per department.

Q3: Find average salary per department.

Q4: Find which department has the highest average salary.

Q5: Count employees per city.

Q6: Find total sales amount per product category.

Q7: Find total sales per month using GROUP BY on the month part of a date.

Q8: List departments having more than 5 employees using HAVING.

Q9: List departments where total salary exceeds 200000 using HAVING.

Q10: Find categories where the average product price is above 1000.

Q11: Count orders per customer, showing only customers with more than 3 orders.

Q12: Find cities that have more than 10 employees.

Q13: Find the maximum salary within each department.

Q14: Find the minimum salary within each department.

Q15: Group sales by category and sort the totals in descending order.

Q16: Find departments with an average salary between 30000 and 60000 using GROUP BY and HAVING.

Q17: Count how many products exist in each category, keeping only categories with more than 5 products.

Q18: Find total quantity sold per product, sorted from highest to lowest.

Q19: Group employees by Department and City together, showing the count for each combination.

Q20: Build a full department report: department name, employee count, total salary, average salary - today's main task.

DAY 21: INNER JOIN (20 Questions)

Q1: Create a Department table (DeptID, DeptName) and link it to Employee through a DeptID column.

Q2: Write an INNER JOIN to show each employee's name along with their department name.

Q3: Join Employee and Department to show only employees who work in the 'IT' department.

Q4: Join Orders and Customers to display the customer name alongside each order.

Q5: Join Products and Categories to display product name alongside category name.

Q6: Find employees and their department names, ordered alphabetically by department.

Q7: Join Employee and Department, then filter for employees earning above 40000.

Q8: Join three tables - Orders, Customers, and Products - to build a complete order detail view.

Q9: Count how many employees exist in each department using INNER JOIN combined with GROUP BY.

Q10: Find total salary per department using INNER JOIN and SUM together.

Q11: Join Employee and Department to list employees whose department name contains 'Sales'.

Q12: Rewrite a JOIN query using table aliases (e for Employee, d for Department).

Q13: Find all employees along with their department name and department location.

Q14: Join Sales and Products to find total revenue generated by each product name.

Q15: Join Customers and Orders to find customers who have placed more than one order.

Q16: Join Employee and Department to identify which department has the most employees.

Q17: Combine INNER JOIN with WHERE and ORDER BY to list the top-paid employee in each department.

Q18: Join Employee and Department, selecting only Name, DeptName, and Salary columns.

Q19: Use INNER JOIN with GROUP BY and HAVING to find departments where more than 2 employees work together.

Q20: Build the complete Employee-Department database showing name, department, city, and salary - today's main task.

DAY 22: LEFT, RIGHT, FULL JOIN (20 Questions)

- Q1:** Write a LEFT JOIN to show all customers, including those who have never placed an order.
- Q2:** Write a RIGHT JOIN to show all orders, including any without a matching customer.
- Q3:** Find customers who have never placed an order using LEFT JOIN with WHERE ... IS NULL.
- Q4:** Find orders that have no matching customer using RIGHT JOIN with WHERE ... IS NULL.
- Q5:** Simulate a FULL OUTER JOIN in MySQL by combining a LEFT JOIN and RIGHT JOIN with UNION.
- Q6:** Use LEFT JOIN to list every product with its total quantity sold, including 0 for unsold products.
- Q7:** Use LEFT JOIN to find employees who are not assigned to any department.
- Q8:** Run the same two tables through INNER JOIN and LEFT JOIN, and compare the row counts.
- Q9:** Use LEFT JOIN to count how many orders each customer has placed, including customers with zero orders.
- Q10:** Use RIGHT JOIN to list departments that currently have no employees.
- Q11:** Use LEFT JOIN to show each customer's most recent order date, or NULL if they have none.
- Q12:** Use LEFT JOIN together with COALESCE to replace NULL order amounts with 0.
- Q13:** Use LEFT JOIN to identify products that have never appeared in a sale.
- Q14:** Use LEFT JOIN to list every category with its product count, including categories with none.
- Q15:** Use LEFT JOIN between Customers and Orders to find customers with no email or phone on file.
- Q16:** Use RIGHT JOIN to find sales records that reference a product no longer in the Products table.
- Q17:** Combine LEFT JOIN with GROUP BY to calculate total spend per customer, including customers with 0 spend.
- Q18:** Write a query that shows every customer name with their order ID, or NULL if there is no order.
- Q19:** Use LEFT JOIN to build a customer activity report: name, total orders, total amount spent.
- Q20:** Build the complete Orders & Customers report using LEFT JOIN so no customer is left out - today's main task.

DAY 23: SELF JOIN, CROSS JOIN (15 Questions)

- Q1:** Create an Employee table that includes a ManagerID column referencing EmpID.
- Q2:** Write a SELF JOIN to display each employee's name alongside their manager's name.
- Q3:** Find employees who have no manager (the top of the hierarchy) using SELF JOIN with IS NULL.
- Q4:** Find all employees who report directly to one specific manager.
- Q5:** Count how many employees report to each manager using SELF JOIN with GROUP BY.
- Q6:** Find pairs of employees who work in the same department using SELF JOIN.
- Q7:** Find employees who earn more than their own manager using SELF JOIN.
- Q8:** Write a CROSS JOIN between Products and Colors to generate every possible product-color combination.
- Q9:** Use CROSS JOIN to create every Employee-Month combination for an attendance tracker.
- Q10:** Compare the row count of an INNER JOIN versus a CROSS JOIN on the same two small tables.
- Q11:** Use SELF JOIN to find employees who were hired in the same city as another employee.
- Q12:** Build a 2-level hierarchy report - employee, manager, and manager's manager - using SELF JOIN twice.
- Q13:** Use CROSS JOIN to generate a price list for every Product-Size combination in a store.
- Q14:** Find duplicate employee names in the table using SELF JOIN.
- Q15:** Build the complete organization hierarchy report using SELF JOIN - today's main task.

DAY 24: Subqueries (20 Questions)

- Q1:** Find the employee with the highest salary using a subquery inside WHERE.
- Q2:** Find employees earning more than the average salary using a subquery.
- Q3:** Find the second-highest salary using a subquery combined with LIMIT and OFFSET.
- Q4:** Find all products priced above the average product price using a subquery.
- Q5:** Find customers who have placed at least one order using a subquery with IN.
- Q6:** Find customers who have never placed an order using a subquery with NOT IN.
- Q7:** Find the department with the highest total salary using a subquery.
- Q8:** Use a subquery inside SELECT to show each employee's salary next to the company-wide average.
- Q9:** Find products that have never been sold using a subquery with NOT IN on the Sales table.
- Q10:** Use a correlated subquery to find employees earning more than the average salary of their own department.
- Q11:** Find the top 3 highest-paid employees using a subquery combined with ORDER BY and LIMIT.
- Q12:** Use EXISTS to find customers who have placed at least one order.
- Q13:** Use NOT EXISTS to find products with zero recorded sales.
- Q14:** Find the lowest-paid employee in each department using a correlated subquery.
- Q15:** Use a subquery to find orders with an amount greater than the average order amount.
- Q16:** Find categories where the average price is higher than the overall average price using a subquery.
- Q17:** Use a subquery to count how many employees earn above the company-wide average salary.
- Q18:** Use a subquery inside FROM to first summarize sales by category, then filter that summary.
- Q19:** Find the customer who has spent the most in total using a subquery.
- Q20:** Find the employee with the highest salary in the entire company, along with their department name - today's main task.

DAY 25: CASE WHEN (15 Questions)

- Q1:** Use CASE WHEN to label employees as 'High', 'Medium', or 'Low' salary based on set thresholds.
- Q2:** Use CASE WHEN to classify customers into 'Young', 'Middle-aged', or 'Senior' based on Age.
- Q3:** Use CASE WHEN to label orders as 'Small', 'Medium', or 'Large' based on Amount.
- Q4:** Use CASE WHEN inside ORDER BY to sort records in a custom priority order.
- Q5:** Use CASE WHEN to convert a numeric marks column into letter grades (A, B, C, F).
- Q6:** Combine CASE WHEN with GROUP BY to count how many employees fall into each salary band.
- Q7:** Use CASE WHEN to flag customers as 'VIP' when their total spend is above a chosen threshold.
- Q8:** Use CASE WHEN to create a Pass/Fail column based on a marks column.
- Q9:** Use CASE WHEN inside SUM() to total two categories separately in a single query (conditional aggregation).
- Q10:** Use CASE WHEN to label products as 'In Stock', 'Low Stock', or 'Out of Stock' based on Stock quantity.
- Q11:** Use a nested CASE WHEN to classify employees by both salary and department together.
- Q12:** Use CASE WHEN to replace NULL values in a column with a readable default label.
- Q13:** Use CASE WHEN to build a discount column: 10% for orders above 5000, 5% otherwise.
- Q14:** Use CASE WHEN to categorize customers by City into 'Metro' or 'Non-Metro'.
- Q15:** Build a complete customer category report using CASE WHEN to segment customers by spend and age - today's main task.

DAY 26: CTE (Common Table Expressions) (15 Questions)

- Q1:** Write a simple CTE that selects all employees earning above 40000, then query from it.
- Q2:** Use a CTE to summarize total sales per category, then filter categories above a chosen threshold.
- Q3:** Use a CTE to first summarize orders per customer, then join it back to the Customers table.
- Q4:** Use a CTE to calculate average salary per department, then compare each employee to their own department average.
- Q5:** Rewrite one of your Day 24 subquery answers using a CTE instead.
- Q6:** Use a CTE to calculate monthly revenue, then find the single month with the highest revenue.
- Q7:** Use two CTEs in one query - one for total sales, one for total returns - then combine them.
- Q8:** Use a CTE to identify the top 5 customers by total spend.
- Q9:** Use a recursive CTE to list an employee hierarchy starting from a top-level manager.
- Q10:** Use a CTE to remove duplicate rows, keeping only the first row per group.
- Q11:** Use a CTE to list products priced above their own category's average price.
- Q12:** Use a CTE to clean and prepare data before applying window functions on top of it.
- Q13:** Use a CTE to break a long, nested Day 24 subquery into clear, readable steps.
- Q14:** Combine a CTE with CASE WHEN to build a customer segmentation report.
- Q15:** Build a sales ranking report using a CTE that ranks products by total revenue - today's main task.

DAY 27: ROW_NUMBER, RANK, DENSE_RANK (20 Questions)

- Q1:** Use ROW_NUMBER() to assign a unique rank to each employee ordered by salary.
- Q2:** Use RANK() to rank employees by salary and note how tied values are handled.
- Q3:** Use DENSE_RANK() on the same data and compare its results with RANK().
- Q4:** Use ROW_NUMBER() with PARTITION BY Department to rank employees within each department.
- Q5:** Find the top 3 highest-paid employees in each department using RANK() and a filter.
- Q6:** Use ROW_NUMBER() to remove duplicate rows, keeping only the first occurrence per group.
- Q7:** Rank products within each category by total sales using RANK().
- Q8:** Use DENSE_RANK() to rank customers by total spend without skipping numbers on ties.
- Q9:** Find the 2nd highest sale in each region using ROW_NUMBER() with PARTITION BY.
- Q10:** Build a leaderboard of top salespeople by total revenue using RANK().
- Q11:** Use ROW_NUMBER() to paginate a large result set, returning only rows 11-20.
- Q12:** Compare ROW_NUMBER(), RANK(), and DENSE_RANK() side by side on the same ordered column.
- Q13:** Rank employees by hire date to find the most senior employee in each department.
- Q14:** Use PARTITION BY City combined with RANK() to find the top earner in each city.
- Q15:** Find employees tied for the highest salary using a filter of RANK() = 1.
- Q16:** Use DENSE_RANK() to assign performance tiers (1st, 2nd, 3rd tier) to salespeople.
- Q17:** Rank orders by amount within each customer using ROW_NUMBER() PARTITION BY CustomerID.
- Q18:** Use ROW_NUMBER() to find the most recent order placed by each customer.
- Q19:** Combine RANK() with a CTE to keep only the top-ranked row per group.
- Q20:** Build a complete sales leaderboard: rank, salesperson name, total revenue - today's main task.

DAY 28: LAG, LEAD (15 Questions)

- Q1:** Use LAG() to show each month's sales alongside the previous month's sales.
- Q2:** Use LEAD() to show each month's sales alongside the next month's sales.
- Q3:** Calculate month-over-month sales growth using LAG().
- Q4:** Use LAG() with PARTITION BY to compare each product's sales to its own previous month.
- Q5:** Use LEAD() to find the next order date placed by each customer.
- Q6:** Use LAG() to find an employee's previous salary from a salary history table.
- Q7:** Calculate the difference between the current and previous day's stock levels using LAG().
- Q8:** Use LAG() with a default value to handle the first row, which has no previous value.
- Q9:** Use LEAD() to spot customers who placed no further orders after a given order (a churn signal).
- Q10:** Combine LAG() and LEAD() in one query to show previous, current, and next month's sales together.
- Q11:** Use LAG() to flag any month where sales dropped compared to the previous month.
- Q12:** Use LAG() with PARTITION BY Region to compare each region's month-over-month performance.
- Q13:** Use LEAD() to calculate the number of days until each customer's next order.
- Q14:** Use LAG() to build a 3-month moving comparison: current month vs previous vs two months ago.
- Q15:** Build a previous month sales comparison report using LAG() - today's main task.

DAY 29: Mixed Revision - All Topics (Mock Interview Prep) (50 Questions)

- Q1:** Write a query to select all columns from a table and explain when `SELECT *` should be avoided.
- Q2:** Write a query using `DISTINCT` on two columns and explain how it differs from one column.
- Q3:** Filter rows using `WHERE` with `AND`, `OR`, and `NOT` combined in one query.
- Q4:** Write a query using `LIKE` to match a pattern at the start, end, and middle of a string.
- Q5:** Explain and demonstrate the difference between `BETWEEN` and using `>=` and `<=` manually.
- Q6:** Write a query using `IN` and rewrite the same logic using multiple `OR` conditions.
- Q7:** Sort a result set by two columns, one ascending and one descending, in the same query.
- Q8:** Use `LIMIT` and `OFFSET` together to fetch a specific 'page' of results.
- Q9:** Explain the difference between `COUNT(*)` and `COUNT(column_name)`.
- Q10:** Write a query that finds the `SUM`, `AVG`, `MIN`, and `MAX` of a column in one `SELECT`.
- Q11:** Explain why aggregate functions require `GROUP BY` when other columns are also selected.
- Q12:** Write a `GROUP BY` query and explain when to use `WHERE` versus `HAVING`.
- Q13:** Write a query with `GROUP BY` and `HAVING` that filters on an aggregated value.
- Q14:** Explain the difference between `INNER JOIN` and `LEFT JOIN` with an example.
- Q15:** Write a query showing unmatched rows using `LEFT JOIN` and `IS NULL`.
- Q16:** Explain when a `RIGHT JOIN` might be preferred over rewriting as a `LEFT JOIN`.
- Q17:** Explain what a `SELF JOIN` is and give a real-world example of when to use one.
- Q18:** Explain what a `CROSS JOIN` produces and when it is actually useful.
- Q19:** Write a subquery inside a `WHERE` clause and explain how it executes.
- Q20:** Explain the difference between a correlated subquery and a regular subquery.
- Q21:** Rewrite a nested subquery as a CTE and explain the readability benefit.
- Q22:** Explain what `EXISTS` does differently compared to `IN`.
- Q23:** Write a query using `CASE WHEN` to bucket a numeric column into 3 categories.
- Q24:** Explain conditional aggregation and write an example using `SUM(CASE WHEN ...)`.
- Q25:** Explain what a Common Table Expression is and how it differs from a temporary table.
- Q26:** Write a query using two CTEs chained together.
- Q27:** Explain what a recursive CTE is used for.
- Q28:** Explain the difference between `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()`.

Q29: Write a query using PARTITION BY to rank rows within groups.

Q30: Explain how to find the 'Nth highest value' using a window function.

Q31: Write a query using LAG() and explain what a NULL in the first row means.

Q32: Explain the difference between LAG() and a self-join for comparing previous rows.

Q33: Write a query combining JOIN, GROUP BY, and HAVING in a single statement.

Q34: Write a query combining a subquery and a window function in a single statement.

Q35: Explain the logical order in which SQL actually processes clauses (FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY).

Q36: Write a query to find duplicate rows in a table.

Q37: Write a query to delete duplicate rows while keeping one copy, using ROW_NUMBER().

Q38: Explain the difference between UNION and UNION ALL.

Q39: Write a query using UNION to combine results from two similar tables.

Q40: Explain what a foreign key is and why JOINS rely on them.

Q41: Write a query to find the top 3 records per group using a window function.

Q42: Explain indexing at a high level and why it speeds up SELECT queries.

Q43: Write a query using COALESCE to handle NULL values in a report.

Q44: Explain the difference between WHERE and HAVING with a concrete example.

Q45: Write a query that returns employees who earn more than their department's average salary.

Q46: Write a query that finds months where revenue increased compared to the previous month.

Q47: Write a query to calculate a running total using a window function.

Q48: Explain a scenario where you would choose a CTE over a subquery, and vice versa.

Q49: Write one query end-to-end that uses at least 4 different concepts from Days 16-28.

Q50: Prepare to explain out loud, in plain English, how any 3 queries above actually work, step by step.

DAY 30: SQL Mini Project - Supermarket Sales Analysis (32+ Questions)

Dataset: Use a Supermarket Sales dataset (freely available on Kaggle) with columns similar to: InvoiceID, Branch, City, CustomerType, Gender, ProductLine, UnitPrice, Quantity, Tax, Total, Date, Time, Payment, Rating.

Q1: Load the Supermarket Sales dataset into MySQL and design an appropriate table schema.

Q2: Find total revenue generated by each branch.

Q3: Find total revenue generated by each product line.

Q4: Find the best-selling product line by total quantity.

Q5: Find the average customer rating per branch.

Q6: Find total sales split by gender.

Q7: Find total sales split by customer type (Member vs Normal).

Q8: Find which payment method is used most often.

Q9: Find total revenue generated by each payment method.

Q10: Find the day of the week with the highest total sales using date functions.

Q11: Find the hour of the day with the most transactions using time functions.

Q12: Find the average unit price per product line.

Q13: Find which branch has the highest average customer rating.

Q14: Find the monthly revenue trend across the full dataset.

Q15: Find the top 5 highest single invoices by total amount.

Q16: Compare the preferred product line for Member customers versus Normal customers.

Q17: Calculate total tax collected per branch.

Q18: Find the average quantity purchased per invoice.

Q19: Rank product lines by total revenue using RANK().

Q20: Use a CTE to calculate revenue per city, then filter to cities above the average.

Q21: Use a subquery to find invoices with a total above the overall average invoice total.

Q22: Use CASE WHEN to bucket ratings into 'Poor', 'Average', 'Good', and 'Excellent'.

Q23: Use LAG() to compare each day's revenue with the previous day's revenue.

Q24: Find whether male or female customers spend more on average per invoice.

Q25: Find which city has the highest number of Member customers.

Q26: Use GROUP BY and HAVING to find product lines with more than 100 transactions.

Q27: Build a summary table: branch, total revenue, total transactions, average rating.

Q28: Use a window function to rank customer types by total spend within each city.

Q29: Combine at least 3 tables or CTEs using JOINS to build one consolidated report.

Q30: Write a final query answering: which branch, product line, and customer type combination generates the most revenue?

Q31: Write 5 short business insights based on what your queries revealed.

Q32: Push all 30+ queries to GitHub as a commented Supermarket_Sales_Analysis.sql file.

Practice Tips & Guidelines

- ✓ **Daily Practice:** Dedicate 30-40 minutes to solve questions for each day.
- ✓ **No Copy-Paste:** Type every query from scratch in MySQL Workbench, don't copy from solutions.
- ✓ **Understand Errors:** When a query fails, read the error message and debug it yourself before searching.
- ✓ **Verify With Small Data:** Test each query against a small sample table so you can eyeball the result.
- ✓ **Query Review:** After solving, check if your query can be simplified or made more readable.
- ✓ **Track Progress:** Mark completed questions and note how long each day took.
- ✓ **Upload to GitHub:** Save your .sql files for each day in your GitHub repository with short comments.
- ✓ **Revision (Day 29):** Treat it like a real interview - explain your query logic out loud as you write it.
- ✓ **Time Management:** Days 16-28 (30-40 min each), Day 29 (60-90 min), Day 30 (90+ min project).
- ✓ **Documentation:** Add a comment above each query explaining what business question it answers.

Progress Summary

Day	Questions	Topics	Difficulty
16	20	SELECT & DISTINCT	Basic
17	25	Filtering (WHERE / LIKE)	Basic
18	20	Sorting & Filtering	Basic → Intermediate
19	20	Aggregate Functions	Intermediate
20	20	GROUP BY & HAVING	Intermediate
21	20	INNER JOIN	Intermediate
22	20	OUTER JOINS	Intermediate
23	15	SELF & CROSS JOIN	Intermediate
24	20	Subqueries	Intermediate → Advanced
25	15	CASE WHEN	Intermediate
26	15	CTEs	Advanced
27	20	Window Functions (RANK)	Advanced
28	15	Window Functions (LAG/LEAD)	Advanced
29	50	Full Revision / Interview Prep	Advanced
30	32	Capstone Project	Advanced

Next Steps: After completing this SQL practice (Days 16-30), you'll be ready for Days 31-45 (Excel, Statistics & Power BI).